FIN 4080 BEHAVIORAL FINANCE

Coding for Momentum Strategy

September 2026

Roadmap

1. Portfolio Sorting
2. Momentum Coding
3. Coding Agent

Recap: What is Asset Pricing?

It studies how financial assets are priced and why their expected returns differ.

Core Question: What stocks have higher return in the future?
Or equivalently: What determines the return an investor requires to hold a risky asset?

- **Expected Return:** compensation for holding risk.
- **Risk Factors:** sources of systematic risk (e.g., market, size, value, momentum).
- **Anomalies:** patterns where returns are predictable from observable characteristics.

Goal: Identify what predicts returns $\Rightarrow$ build better portfolios.

The Problem: Too Many Stocks, Too Many Characteristics

There are thousands of stocks, each with many characteristics (size, profitability, . . .).

- We cannot simply compare every stock individually.
- Returns of individual stock are very noisy.

Key insight: Group stocks with *similar characteristics* together, then compare their average returns.

What is Portfolio Sorting?

Form portfolios by ranking stocks on a characteristic and grouping them into buckets.

Steps:

1. Pick a characteristics or signal (e.g., past 6-month return).
2. Before time t, rank all stocks by the signal.
3. Split into N groups (e.g., deciles: 1 to 10).
4. Compute the average return of each group in time t.
5. Compare the top group vs. the bottom group.

Result: A "spread portfolio" (e.g., top decile minus bottom decile) that reveals whether the signal predicts returns.

Example: Sorting by Momentum

Portfolio	Average Monthly Return
1 (Losers)	0.59%
2	1.07%
3	1.35%
⋮	⋮
10 (Winners)	1.79%
10 − 1	**1.20%**

Interpretation: The monotonic pattern suggests momentum predicts returns.

Question: What is the "10 − 1" portfolio? What are pros and cons of sorting into 10 vs. 5 portfolios?

Why Do We Need Portfolio Sorting?

- **Simplicity:** No regression needed—just averages.
- **Robustness:** Non-parametric; no assumption about the functional form.
- **Economic Interpretation:** Directly translates into a tradable strategy.
- **Common Language:** Standard tool in empirical asset pricing (Fama–French, Jegadeesh–Titman, etc.).

It answers: Does characteristic X predict returns, and by how much?

From Sorting to a Tradable Strategy

A strategy is useful only if it can be traded.

Requirements for a tradable strategy:

- **Information available at time** t to form portfolios.
- **Returns realized after time** t to evaluate performance.
- **Realistic constraints:** transaction costs, short-selling, liquidity.

Look-Ahead Bias

Look-ahead bias = using information *not actually available* at the time of the trade.

Examples:

- Using **next-month** data to trade in this month.
- Using **restated** accounting data (not the original figure).
- Using **survivorship-biased** data (only stocks that still exist).

Consequence: Your backtest looks profitable, but the strategy fails in real trading.

Rule of thumb: At time t, you may only use information available up to t.

Summary: Portfolio Sorting Workflow

Step	Action
1	Choose a signal (e.g., momentum)
2	Form portfolios by ranking on the signal
3	Compute next-period returns for each portfolio
4	Compare top vs. bottom (spread portfolio)
5	Ensure no look-ahead bias
6	Check robustness (horizons, holding periods)

Next: We apply this workflow to a real momentum strategy.

Momentum Strategy: Jegadeesh & Titman (1993)

The Big Idea: Stocks that performed well (bad) in the past 6 months tend to *continue* performing well (bad) in the near future.

How the Strategy Works:

- Rank all stocks by their past 6-month returns.
- **Buy** the top 10% (the "winners"), and **sell** the bottom 10% (the "losers").
- Hold the portfolio for the next 6 months.

The Result: "Winner-minus-loser" portfolio earned **1% per month** in excess returns.

Why It Matters: It contradicts the Efficient Market Hypothesis.

Momentum Strategy: U.S. vs. China

The Evidence:

- **U.S.:** Past winners continue to outperform past losers.
- **China:** Monthly long-short portfolios earn near-zero or insignificant returns

Why Momentum Doesn't Exist in China

- Small-cap shell-value distortions (Liu, Stambaugh & Yuan, 2019).
- T+1 trading rule (Bai et al., 2020).
- Retail-dominated trading and noise (Chui, Subrahmanyam & Titman, 2022).

Why It Matters: Momentum is not a universal law; institutions and investor composition shape whether it survives.

Step 1. Input Dataset

```
import numpy as np
import pandas as pd

crsp = pd.read_csv("crsp_msf.csv")
crsp.columns = crsp.columns.str.lower()

crsp = crsp.loc[
    crsp["date"].dt.year.between(1965, 1989)
]

crsp.head(10)
```

Step 1. Input Dataset

```
import numpy as np
import pandas as pd

crsp = pd.read_csv("crsp_msf.csv")
crsp.columns = crsp.columns.str.lower()

crsp = crsp.loc[
    crsp["date"].dt.year.between(1965, 1989)
]

crsp.head(10)
```

permno	date	ret
10006	1965-01-31	0.0535
10006	1965-02-28	-0.0260
10006	1965-03-31	0.0402
10006	1965-04-30	0.0848
10006	1965-05-31	-0.0436
10006	1965-06-30	-0.1043
10006	1965-07-31	0.0097
10006	1965-08-31	0.0596
10006	1965-09-30	-0.0122
10006	1965-10-31	0.1084
	.....	

Step 2. Compute Momentum

```
crsp["grossret"] = (1 + crsp["ret"]).where
    (crsp["ret"] > -1)

crsp["mom6"] = (
    crsp["grossret"].groupby(crsp["permno"
    ])
    .rolling(6, min_periods=6)
    .apply(np.prod, raw=True).sub(1)
    .reset_index(level=0, drop=True)

    .groupby(crsp["permno"])
    .shift(2)
)

crsp[4:14]
```

Step 2. Compute Momentum

```
crsp["grossret"] = (1 + crsp["ret"]).where
    (crsp["ret"] > -1)

crsp["mom6"] = (
    crsp["grossret"].groupby(crsp["permno"
    ])
    .rolling(6, min_periods=6)
    .apply(np.prod, raw=True).sub(1)
    .reset_index(level=0, drop=True)

    .groupby(crsp["permno"])
    .shift(2)
)

crsp[4:14]
```

permno	date	grossret	mom6
10006	1965-05-31	0.9564	NaN
10006	1965-06-30	0.8957	NaN
10006	1965-07-31	1.0097	NaN
10006	1965-08-31	1.0596	-0.0081
10006	1965-09-30	0.9878	-0.0493
10006	1965-10-31	1.1084	0.0342
10006	1965-11-30	0.9905	-0.0180
10006	1965-12-31	1.0769	0.0033
10006	1966-01-31	1.0899	0.0391
10006	1966-02-28	0.9796	0.2495
	.....		

Step 3. Rank Stocks by Momentum

```
signal = crsp.loc[
    crsp["shrcd"].isin([10, 11])
    & crsp["exchcd"].isin([1, 2])
    & crsp["mom6"].notna(),
    ["permno", "date", "mom6", "ret"]
]

crsp["rank"] = signal.groupby("date")["
    mom6"].transform(
    lambda x: pd.qcut(x, 10, labels=False,
     duplicates="drop") + 1
)

crsp[4:14]
```

Step 3. Rank Stocks by Momentum

```
signal = crsp.loc[
    crsp["shrcd"].isin([10, 11])
    & crsp["exchcd"].isin([1, 2])
    & crsp["mom6"].notna(),
    ["permno", "date", "mom6", "ret"]
]

crsp["rank"] = signal.groupby("date")["
    mom6"].transform(
    lambda x: pd.qcut(x, 10, labels=False,
     duplicates="drop") + 1
)

crsp[4:14]
```

permno	date	mom6	rank
10006	1965-05-31	NaN	NaN
10006	1965-06-30	NaN	NaN
10006	1965-07-31	NaN	NaN
10006	1965-08-31	-0.0081	4
10006	1965-09-30	-0.0493	4
10006	1965-10-31	0.0342	6
10006	1965-11-30	-0.0180	5
10006	1965-12-31	0.0033	5
10006	1966-01-31	0.0391	5
10006	1966-02-28	0.2495	6
	.....		

Step 4. Average Return by Month-Rank

```
cohort = crsp.groupby(["date", "rank"])["ret"].mean()
port_ret = cohort.groupby(["date", "rank"]).mean().unstack("rank")
port_ret["10-1"] = port_ret[10] - port_ret[1]

port_ret.head()
```

Step 4. Average Return by Month-Rank

```
cohort = crsp.groupby(["date", "rank"])["ret"].mean()
port_ret = cohort.groupby(["date", "rank"]).mean().unstack("rank")
port_ret["10-1"] = port_ret[10] - port_ret[1]

port_ret.head()
```

date	1	2	3	4	5	6	7	8	9	10	10-1
1965-08-31	0.033	0.037	0.051	0.036	0.036	0.029	0.037	0.048	0.051	0.060	0.027
1965-09-30	-0.011	0.036	0.018	0.019	0.028	0.028	0.024	0.037	0.039	0.061	0.073
1965-10-31	0.069	0.054	0.048	0.045	0.047	0.036	0.053	0.070	0.078	0.090	0.021
1965-11-30	0.035	0.044	0.025	0.025	0.045	0.034	0.056	0.054	0.068	0.092	0.058
1965-12-31	0.064	0.056	0.045	0.043	0.043	0.031	0.036	0.038	0.039	0.033	-0.032

Step 5. Average Portfolio Return

```
port_mean = port_ret.mean(axis=0)
port_tstat = port_mean / (
    port_ret.std(axis=0, ddof=1)
    / np.sqrt(port_ret.count(axis=0))
)

port_summary = pd.DataFrame({
    "mean": port_mean,
    "t_stat": port_tstat,
})

port_summary
```

Step 5. Average Portfolio Return

```
port_mean = port_ret.mean(axis=0)
port_tstat = port_mean / (
    port_ret.std(axis=0, ddof=1)
    / np.sqrt(port_ret.count(axis=0))
)

port_summary = pd.DataFrame({
    "mean": port_mean,
    "t_stat": port_tstat,
})

port_summary
```

rank	mean	t_stat
1	0.005873	1.130615
2	0.010745	2.540310
3	0.013533	3.436807
4	0.013610	3.696711
5	0.013390	3.853567
6	0.013904	4.100951
7	0.014467	4.287097
8	0.014968	4.287773
9	0.015682	4.309555
10	0.017864	4.333754
10-1	0.011990	3.533812

Step 5. Plotting Portfolio Returns

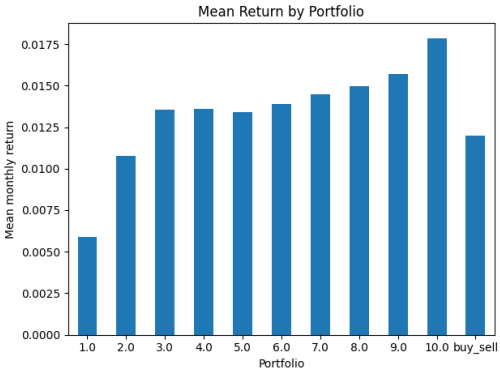
```
import matplotlib.pyplot as plt

port_summary["mean"].plot(kind="bar")
plt.xlabel("Portfolio")
plt.ylabel("Mean monthly return")
plt.title("Mean Return by Portfolio")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

Step 5. Plotting Portfolio Returns

```
import matplotlib.pyplot as plt

port_summary["mean"].plot(kind="bar")
plt.xlabel("Portfolio")
plt.ylabel("Mean monthly return")
plt.title("Mean Return by Portfolio")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



What is a Coding Agent?

An AI system that can *autonomously* write and run code to accomplish a task.

Key Difference from a Chatbot:

- **Chatbot:** You ask, it answers. You copy-paste and run.
- **Coding Agent:** You describe a goal. It plans, writes code, debugs, and executes.

Examples: GitHub Copilot, Claude Code, OpenAI Codex.

Variation 1. Month After Portfolio Formation

We report returns that are 2 months after portfolio formation
We can also report returns that are 1/2/3/.../36 month after formation

Let Coding Agent do the work:
‘‘‘
After creating the signal and sorting the portfolio, report the average returns for the long-short portfolio for 1 month after formation, 2 month, ... up until 36 month after the portfolio formation. Return mean and t-statistics for each month.
’’’

Variation 1. Month After Portfolio Formation

t	Monthly Return	Cumulative Return	t	Monthly Return	Cumulative Return	t	Monthly Return	Cumulative Return
1	−0.0025	−0.0025	13	−0.0036	0.0915	25	−0.0035	0.0521
	(−0.59)	(−0.59)		(−1.12)	(3.35)		(−1.36)	(1.41)
2	0.0124	0.0099	14	−0.0039	0.0876	26	−0.0030	0.0492
	(3.29)	(1.37)		(−1.34)	(3.07)		(−1.14)	(1.22)
3	0.0116	0.0216	15	−0.0034	0.0842	27	−0.0024	0.0467
	(3.18)	(2.20)		(−1.21)	(2.89)		(−0.98)	(1.10)
4	0.0110	0.0326	16	−0.0038	0.0804	28	−0.0032	0.0435
	(3.19)	(2.67)		(−1.41)	(2.76)		(−1.33)	(0.98)
5	0.0093	0.0419	17	−0.0047	0.0757	29	−0.0032	0.0403
	(2.82)	(2.79)		(−1.74)	(2.70)		(−1.38)	(0.87)
6	0.0091	0.0510	18	−0.0056	0.0701	30	−0.0030	0.0373
	(2.94)	(2.92)		(−2.19)	(2.68)		(−1.31)	(0.77)
7	0.0134	0.0644	19	−0.0026	0.0675	31	−0.0001	0.0372
	(4.98)	(3.32)		(−1.14)	(2.75)		(−0.06)	(0.74)
8	0.0115	0.0759	20	−0.0032	0.0642	32	0.0008	0.0380
	(4.16)	(3.60)		(−1.35)	(2.73)		(0.41)	(0.73)
9	0.0085	0.0844	21	−0.0032	0.0611	33	0.0013	0.0394
	(3.07)	(3.73)		(−1.32)	(2.55)		(0.62)	(0.73)
10	0.0048	0.0892	22	−0.0034	0.0577	34	0.0008	0.0402
	(1.69)	(3.74)		(−1.39)	(2.21)		(0.36)	(0.71)
11	0.0045	0.0938	23	−0.0011	0.0566	35	0.0010	0.0412
	(1.55)	(3.77)		(−0.45)	(1.93)		(0.45)	(0.71)
12	0.0013	0.0951	24	−0.0010	0.0556	36	−0.0005	0.0406
	(0.43)	(3.67)		(−0.40)	(1.69)		(−0.24)	(0.67)

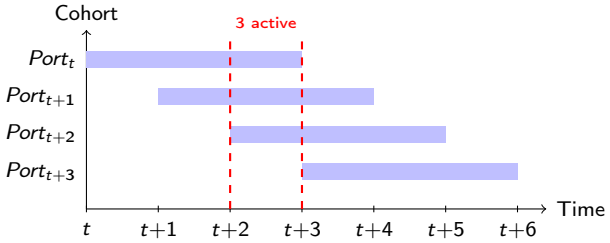
Variation 2: Horizon

Computing Momentum

- We compute momentum using cumulative returns over the past 6 month

Future Returns

- We hold the long-short portfolio for one month after portfolio formation.
- We can also hold the long-short portfolio for multiple months.



Variation 2. Horizon

Let Coding Agent do the work:
‘‘‘
1. Create momentum signal based on returns in past 3/6/9/12 month
2. Hold each portfolio from t+1 to t+3/t+6/t+9/t+12 month. Then each month I will be holding 3/6/9/12 portfolios with equal weight
3. Compute the average returns for this strategy. Report mean and t-statistics only for loser, winner, and long-short portfolio.
4. Keep 4 columns only, each representing holding period for 3/6/9/12 month
’’’

Variation 2. Horizon

		Panel A			
J	$K=$	3	6	9	12
3	Sell	0.0108	0.0091	0.0092	0.0087
		(2.16)	(1.87)	(1.92)	(1.87)
3	Buy	0.0140	0.0149	0.0152	.0156
		(3.57)	(3.78)	(3.83)	(3.89)
3	Buy-sell	0.0032	0.0058	0.0061	0.0069
		(1.10)	(2.29)	(2.69)	(3.53)
6	Sell	0.0087	0.0079	0.0072	0.0080
		(1.67)	(1.56)	(1.48)	(1.66)
6	Buy	0.0171	0.0174	0.0174	0.0166
		(4.28)	(4.33)	(4.31)	(4.13)
6	Buy-sell	0.0084	0.0095	0.0102	0.0086
		(2.44)	(3.07)	(3.76)	(3.36)
9	Sell	0.0077	0.0065	0.0071	0.0082
		(1.47)	(1.29)	(1.43)	(1.66)
9	Buy	0.0186	0.0186	0.0176	0.0164
		(4.56)	(4.53)	(4.30)	(4.03)
9	Buy-sell	0.0109	0.0121	0.0105	0.0082
		(3.03)	(3.78)	(3.47)	(2.89)
12	Sell	0.0060	0.0065	0.0075	0.0087
		(1.17)	(1.29)	(1.48)	(1.74)
12	Buy	0.0192	0.0179	0.0168	0.0155
		(4.63)	(4.36)	(4.10)	(3.81)
12	Buy-sell	0.0131	0.0114	0.0093	0.0068
		(3.74)	(3.40)	(2.95)	(2.25)